

三个开源项目学习总结

本笔记目前整理到：Epftoolbox（电价预测基准库），EFM3（生产链路与审计机制）与 FutureBoosting 的部分

目录

Epftoolbox: 怎么进行电价预测.....	5
1.1 DNN 模型.....	5
1.1.1 epftoolbox 中的 DNN.....	7
1.1.2 超参数搜索（一种调参方法）.....	8
1.2 LEAR 模型.....	9
1.2.1 数据变换.....	10
1.2.2 训练：先做正则化.....	10
1.2.3 预测.....	11
1.2.4 逐日重校准 + 回测循环（防泄漏）.....	11
1.3 数据集.....	12
1.4 评估指标.....	13
1.4.1 MAE:平均绝对误差.....	13
1.4.2 sMAPE:对称平均绝对百分比误差.....	13
1.4.3 R^2 :决定系数.....	13
1.4.4 MSE:均方误差.....	13
1.4.5 RMSE:均方根误差.....	13
1.4.6 RMSE/MAE（经验判据，非项目实现）.....	14
1.5 结果.....	14
EFM3: 怎么投入生产.....	14
2.1 领域约定：小时映射与分时段.....	14
2.2 数据摄入：回填与幂等.....	15
2.3 台账与血缘.....	19

2.3.1	台账与血缘是什么.....	19
2.3.2	为什么要做台账和血缘.....	20
2.3.3	23 张表分五组.....	20
2.3.4	最核心的一张表: efm_predictions.....	21
2.3.5	血缘怎么连起来: 三种机制	21
2.3.6	一次运行往台账里写了什么 (时序)	23
2.3.7	血缘图怎么算出来的: 不入库, 读时现算.....	23
2.3.8	期望改进	24
2.3.9	分时段硬切可能导致人为断层.....	25
2.4	日前链路: 三模型融合 + 权重学习	26
2.4.1	阶段一: 取模型原始输出.....	27
2.4.2	阶段二: 修复 (三条守卫规则)	27
2.4.3	阶段三: 融合: fusion_chain.py.....	28
2.4.4	阶段四: 分类器 (日前是透传)	28
2.4.5	阶段五: 定稿.....	28
2.4.6	关键补充: 仓库里存在两套融合实现.....	29
2.5	实时链路: 选择器与回退.....	29
2.6	极值处理与防泄漏.....	30
2.7	编排、状态机与交付校验.....	31
2.8	指标与审计口径	32
	FutureBoosting: 零样本预测特征 + 预测电价	32
3.1	项目目标.....	32
3.1.1	时序基础模型	32
3.1.2	真实存在的问题	33

3.1.3 设计细节.....	34
3.1.4 程序入口.....	34
3.2 数据入口与数据格式.....	34
3.2.1 原理一：变量必须按“签发时刻的可得性”分三类.....	34
3.2.2 原理二：框架不造特征，只认“已经算好的列”；TSFM 是唯一的例外....	35
3.2.3 原理三：粒度决定滞后步长也决定模型能看到什么周期	35
3.2.4 真实结果与说明(暂留)	36
3.3 数据清洗.....	36
3.4 时间协议.....	36
3.4.1 切分方式.....	36
3.4.2 回测形态.....	37
3.4.3 与生产的差距	37
3.5 建模与评价	37
3.5.1 baseline.....	37
3.5.2 模型.....	37
3.5.3 训练.....	38
3.6 工程与生产	38

Epftoolbox: 怎么进行电价预测

1.1 DNN 模型

原理：学习大量样本的输入与输出特征之间的关系，以拟合输入与输出特征之间的方程，学习完成后，只要给它一次输入就可以得到对应输出。

步骤为：划分数据集，训练网络，测试网络，使用网络。

1.划分数据集：

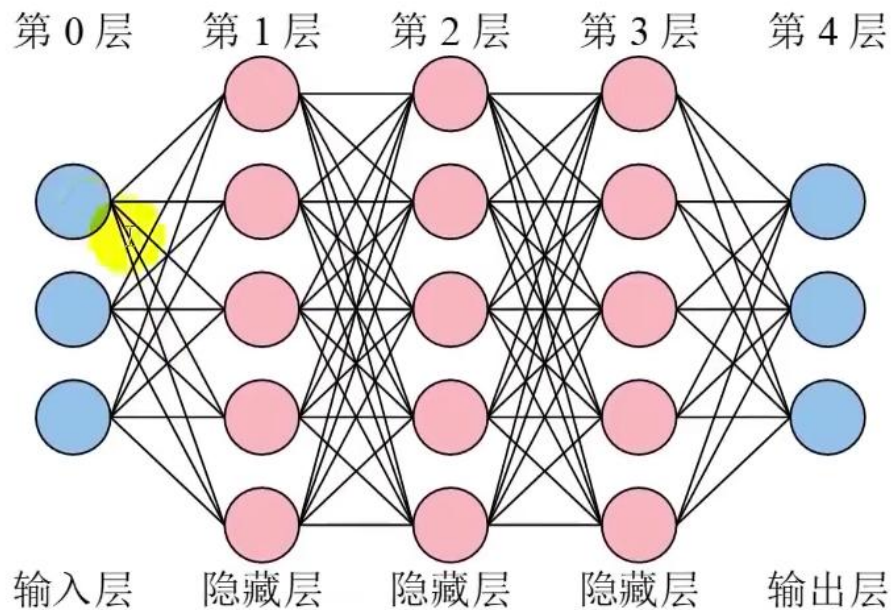
表 2-1 数据集的划分

	样本	输入特征			输出特征		
		In1	In2	In3	Out1	Out2	Out3
训练集	1	*	*	*	*	*	*
	2	*	*	*	*	*	*
	3	*	*	*	*	*	*
	4	*	*	*	*	*	*
	5	*	*	*	*	*	*
	...	*	*	*	*	*	*
	800	*	*	*	*	*	*
测试集	801	*	*	*	*	*	*
	802	*	*	*	*	*	*
	...	*	*	*	*	*	*
	1000	*	*	*	*	*	*

数据集里的每个样本必须包含输入与输出特征，将其划分为训练集和测试集

根据上图，输入与输出特征各为三个，因此输入与输出层均有三个神经元。输入层没有内部参数，只有中间的隐藏层和输出层才有内部参数。

可见共有五层隐藏层，每层三个神经元



2.训练网络:

其训练过程就是经过多次的前向传播和反向传播，最终不断优化其内部参数(权重 w 和偏置 b)，以拟合任意函数的过程。当然还有一些训练前需要设置好的参数，比如每个结点激活函数类型，学习率，轮回次数等。这些外部参数统称为超参数，而内部函数称为参数。

- 前向传播：将单个样本的 n 个输入特征输入到输入层后，神经网络会逐层计算到输出层，最终得到神经网络预测的三个输出特征。

神经元，如图 2-2 所示。

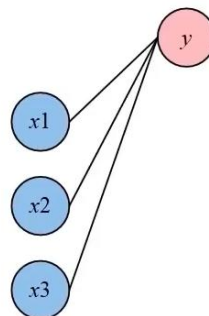
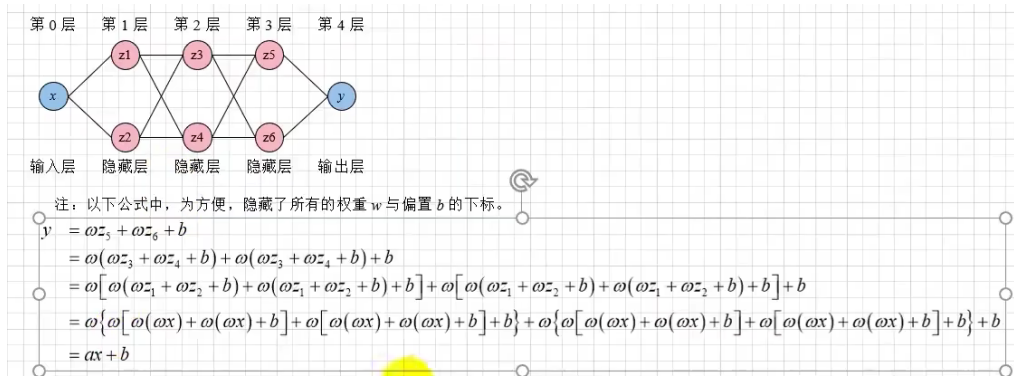


图 2-2 每个神经元节点的计算

该神经元节点的计算过程为 $y = \omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + b$ 。你可以理解为，每一根线就是一个权重 ω ，每一个神经元节点也都有它自己的偏置 b 。

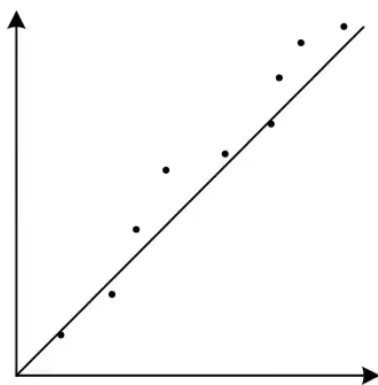
当然，每个神经元节点在计算完后，由于这个方程是线性的，因此必须在外套一个非线性的函数： $y = \sigma(\omega_1 x_1 + \omega_2 x_2 + \omega_3 x_3 + b)$ ， σ 被称为激活函数。如果你不套非线性函数，那么即使 10 层的网络，也可以用 1 层就拟合出同样的方程。

为什么要激活函数：线性方程经过线性变化加偏置后仍然是线性的

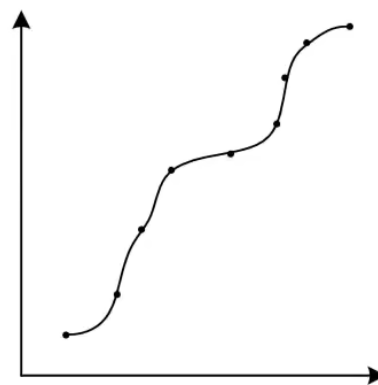


- 反向传播：根据预测值与真实值计算损失函数，通过调整内部参数看损失函数变大还是变小，以此达到优化参数的效果。
- Batch_size：把样本分为若干批，batch_size 即一批样本中的样本数(小批量梯度下降)
- Epochs：1 个 epoch 指的就是完成一次前向和反向传播

3.测试网络：为了防止模型过拟合：即优化好的内部参数只对训练样本有效，换成其他的效果很差，所以需要拿出一部分样本用于测试效果



(a) 正确的拟合



(b) 过拟合

4.使用网络

1.1.1 epftoolbox 中的 DNN

输入神经元个数：五个内置数据集神经元个数为 247；辽宁数据集神经元个数为 175.

计算方式：四个滞后日*24+7 个星期哑变量+72*外生变量数；这些是上限，最终真正放进网络里的有由搜索结果决定。

输出神经元共有 24 个，对应 24 小时。一个网络可以同时输出 24 个小时的价格。输出层是连续的函数，具有外推能力，当输入范围外的输入值时，会根据原有的数学关系适当外推，但是 LEAR 只能沿固定斜率延长直线，没有曲率(DNN 比 LEAR 本质的升级：线性只能直线外推，非线性可以带曲率外推)，因此可以更好地拟合尖峰和负电价，而树模型没有外推能力。

这是与 LEAR 最大的不同：模型类不同(非线性 vs 线性)+单网络多输出。

早停：即如果一定轮次内模型性能未得到的改善(meas 值不再减小)，则停止。

数据变换不做会怎么样：

- (1). 梯度尺度差异：收敛慢/震荡/发散；
- (2). sigmoid/tanh 饱和：梯度消失；
- (3). 直接回归 1500 量级目标，初始化与学习率难设；
- (4). 早停判据被大尺度主导

1.1.2 超参数搜索（一种调参方法）

DNN 把变换之后的 scalerX 和 scalerY 作为超参去搜索

- 为什么这么做：该项目采用贝叶斯优化。先要知道：参数是模型学出来的，而超参数是人为指定的。因此超参数是模型自己学不出来的，没有公式可解，只能够搜出来，而 epftoolbox 采用贝叶斯优化在有限次搜索内找到足够好的配置，还顺便决定用哪些特征，而代价就是搜索很昂贵，一次搜索可能耗时几天，而本项目中作者自己花算力把五个市场的搜索结果随库发布。

这也解释了为什么 experimental_files/里有那么多

DNN_hyperparameters_nl2_datPJM_YT2_SF_SF_DA_CW4_1 之类的文件

用这个项目来说就是：搜索一次性决定了一定时间内网络结构，训练方式，用哪些特征，在这段时间内避免了盲目调参，但仍需要注意的是需要每天重新训练一个网络(但是结构已经暂时定下来了)，权重从零学起；另外由于电力市场结构可能会发生变化，因此每隔一段时间后，原先已经决定好的最优配置可能不再是最优，需要重新进行搜索。

- 如果去掉后会发生什么：网络结构随便定，过大导致过拟合，过小导致欠拟合；学习率随便定，小了训练耗时，大了可能会发散；跨市场用同一套导致偏差明显等。
- 怎么做的：用 TPE 反复进行“采样一组配置→每次评估都要训练一个 DNN→用验证集 MAE 打分→反馈给采样器→根据历史分数决定下一组采样”。(TPE 是贝叶斯优化的一种具体实现方法)

TPE:

- 解决什么问题：超参数空间巨大，每次训练一个网络又很昂贵，所以需要有一个适当的方法。
- 怎么做的：将已经试过的配置按 meas 大小排序，取最后的前一部分分为“好组”，后部分分为“差组”；

第 2 步：分别给两组估一个概率分布

用核密度估计（KDE，也就是 "Parzen 窗"）分别拟合：

- $l(x)$ = "在好组里，配置 x 出现的概率密度"
- $g(x)$ = "在差组里，配置 x 出现的概率密度"

第 3 步：下一个采样点，选让 $l(x)/g(x)$ 最大的那个 x

1.2 LEAR 模型

名字与本质：LEAR = Lasso Estimated AutoRegressive

AutoRegressive：自回归，即用自己过去的值预测未来的值—用历史电价预测未来电价

Lasso Estimated：Lasso 估计，用 L1 正则化回归拟合，自动把无用特征系数压缩到 0

关键：每小时各构建一个模型，一共 24 个。

为什么：因为凌晨三点和晚高峰 19 点电价形成机制不同。"机制不同"不只是"影响程度不同"，还包括影响方向、主导变量等都不同；LEAR 用"每小时一个模型"来近似这种异质性，但受限于线性假设只能画直线，这正是 DNN（非线性）和 EFM3（分时段 + 尖峰分类器）要解决的问题。

输入特征：与 DNN 一致：

$$\text{输入特征数} = 96 + 72 \times \text{外生变量数} + 7$$

其中 72 为每个外生变量的 D-1,D-7 以及预测日的 24 点

为什么要选择 D-1,D-7 以及预测日：注意与价格区分，价格有"惯性"，昨天涨了今天还可能涨，所以需要连续的近几天；而外生变量是"物理状态"，今天的负荷，光伏出电等主要取决于天气和作息，跟季节和星期关系大；其中 D(当天)是最准确的；

但区别是：DNN 中不是把全部特征全部喂给模型，而是由超参数搜索决定哪些特征放入；而 LEAR 是全部特征全部放入，由 Lasso 决定哪些系数归零，就是不要。

为什么价格滞后要选 D-1,D-2,D-3,D-7，不选 D-1-D-7：因为电价自相关集中在邻近天和一周节律，D4-D6 信息价值太低，放进去也只会当作噪声。

1.2.1 数据变换

解决什么问题：

- (1). 量纲差异太大，电价几百元/MWh，甚至可能是负数；系统负荷几万 MW；风速等可能是个位到几百。对于线性模型是不公平的正则化；而对于神经网络是失衡的梯度，学习率难以统一。
- (2). 电价分布不均，有尖峰，有负电价，导致带偏整个模型。

怎么解决的：

- (1). ArcSinh 做尾部压缩变换：目的是将尖峰和负电价进行压缩(不用 log 是因为 log 无法处理负值)
- (2). 中位数/MAD 代替均值/标准差：中位数，MAD 对于极端电价更稳健

$$z = \operatorname{arcsinh}((x - \text{median}) / \text{MAD})$$

哑变量不缩放，本身 0/1 硬标签无量纲，缩放后反而会破坏含义

- 如何去掉变换会怎么样：

- (1). 正则化不公平(LEAR)：特征选择结果被尺度扭曲（Lasso 可能保住大尺度特征、误杀小尺度特征）；
- (2). 极端值主导 MSE：模型被尖峰带偏；
- (3). LARS/Lasso 数值路径不稳定、收敛变慢

总结：由于树模型只关心排序，线性模型和神经网络需要考虑到正则化和梯度因此需要对数据进行变换。数据变换发生在数据前预处理步骤。是否变换取决于模型类型。但数据变换带来的代价是最后需要进行逆变换才是真实的价格。

$$\text{逆变换: } x = \sinh(z) \times \text{MAD} + \text{median}$$

1.2.2 训练：先做正则化

- 用 LARS 路径 + AIC 准则自动估出最优 α ，因此 LEAR 不用调参
- 用这个 α 正式拟合 lasso；
- 解决的问题是 lasso 的关键超参数 α 怎么定

什么是 LARS 路径 + AIC 准则：

一句话概括：用 LARS 算法一次性算出整条正则化路径，再用 AIC 准则从这条路径上挑选最合适的那个 α (免调试，快，可复现)

为什么能够一次性算出：LARS 从系数全 0 出发，每一步沿当前残差相关性最大的方向前进，直到另一个特征的相关性追上；每转一次弯就有一个新特征引入，即对应一个不同的 α 。所以走完整条路径就得到了所有 α 的解。

- LARS 算法是什么：即最小角回归算法，把 α 的解一次性算出来
- AIC 准则是什么：即赤池信息准则，在这些解里把最好的 α 挑出来

$$AIC = n \cdot \log(RSS / n) + 2k$$

RSS 是残差平方和，用于衡量拟合得准不准，k 为非零系数的个数，衡量模型有多复杂，2k 为惩罚项；既要拟合得好，又要系数少，AIC 最小的那个点就是性价比最高的 α ；因此 LEAR 的免调参就是因为 α 是算出来的而非试出来的

```
for h in range(24):
    # ① 选  $\alpha$ : LARS 走完整条路径，用 AIC 挑出最优  $\alpha$ 
    param_model = LassoLarsIC(criterion='aic', max_iter=2500)
    param = param_model.fit(Xtrain, Ytrain[:, h]).alpha_

    # ② 定下  $\alpha$  后，正式拟合一次 Lasso
    model = Lasso(max_iter=2500, alpha=param)
    model.fit(Xtrain, Ytrain[:, h])
    self.models[h] = model
```

- 解决什么问题：传统做法下成本高(候选数*k 次训练)，需要每天重选 α 成本大，人工设置 α 不可复现，求出每个 α 需要网格搜索+k 次训练(k 折交叉验证)

如果去掉后会怎么样：

- 改用网格搜索+交叉验证，虽然结果可能差不多但成本更高，更耗时间
- 固定一个 α ：太大时：系数全为 0，模型退化成永远预测均值；太小时：96 个高相关滞后特征导致严重多重共线性
- 不进行正则化：训练误差更低但是样本外更差—引入 lasso 的原因

1.2.3 预测

数据变换后进行预测前为什么只能用训练时候保存的 scaler：

(scaler 是模型一部分，需要和权重一样一起保存一起使用)

因为这时模型学到的系数是在已经经过变换后的坐标系里成立，也就是说 lasso 看到的不是原始数据而是变换后的数据，若想要真正的结果需要进行逆变换。

```
# _lear.py: predict()
X_no_dummies = self.scalerX.transform(X[:, :-7]) # ① 用训练时的 scalerX 变换输入
X[:, :-7] = X_no_dummies
for h in range(24):
    Yp[h] = self.models[h].predict(X) # ② 24 个模型在 z 空间预测
Yp = self.scalerY.inverse_transform(Yp.reshape(1, -1)) # ③ 用训练时的 scalerY
```

1.2.4 逐日重校准 + 回测循环（防泄漏）

LEAR 和 DNN 用同一个框架，只是模型本体不同

逐日重校准：算出模型新的系数，让模型跟上市场，放在 DNN 里则是更新权重

```
df_train = df.loc[:next_day_date - pd.Timedelta(hours=1)]
df_train = df_train.iloc[-calibration_window * 24:] # 只取最近 N 天
df_test = df.loc[next_day_date - pd.Timedelta(weeks=2):, :] # 多取两周, 用于造 D-7 滞后

Xtrain, Ytrain, Xtest = self._build_and_split_XYs(df_train, df_test, date_test=next_day_date)
Yp = self.recalibrate_predict(Xtrain, Ytrain, Xtest)
```

一些细节:

细节	内容
① 训练窗口	LEAR 用“最近 <code>calibration_window</code> 天”; DNN 是“最近 <code>calibration_window × 364</code> 天”, 单位是“年” (文件名的 CW4 = 4 年)
② 每天的数据	训练集 = 历史 + 测试期到预测日前一天; 再截取最近 N 天
③ 为什么要多取两周	因为要构造 D-7 滞后, 测试集必须从“预测日 - 2 周”开始取
④ 打印的是累积指标	控制台那行 sMAPE/MAE 是“到当前日期为止所有天”的累计值, 不是当天单独指标 (数字会随天数增加逐渐稳定)
⑤ 每轮都落盘	<code>forecast.to_csv()</code> 在循环内部, 中断也有部分结果; DNN 还支持断点续跑 (只补 NaN 日期, LEAR 没有)
⑥ 验证集差异	LEAR 无验证集 (AIC 定 α); DNN 内部切 25% 做验证集用于早停
⑦ 文件命名	LEAR <code>forecast_datX_YT2_CW330.csv</code> ; DNN 是 <code>DNN_forecast_n12_datX_YT2_SFH1_CW4_1.csv</code> (nl=层数, CW=校准窗口、末尾=experiment_id)

回测循环

把当天价格设置为 np.nan 目的是为了防止未来泄露, 因为我预测的就是当天价格, 如果不设置相当于把当天要预测价格也当成训练集, 结果必定相当准确, 起不到预测的意义;

那为什么预测日当天的外生变量不会造成未来泄露: 因为当天的外生变量是已经知道的。已经预测出来的; 而当天的价格是需要进行预测的

```
for date in forecast_dates:
    data_available = pd.concat([df_train, df_test.loc[:date + pd.Timedelta(hours=23), :]], axis=0)

    # ★ 防泄露: 把预测日当天的真实价格抹成 NaN, 模型看不到“答案”
    data_available.loc[date:date + pd.Timedelta(hours=23), 'Price'] = np.nan

    Yp = model.recalibrate_and_forecast_next_day(df=data_available, next_day_date=date,
                                                calibration_window=calibration_window)

    forecast.loc[date, :] = Yp

    # ★ 打印的是“累积指标” (到当前日期为止的所有天), 不是当天单独指标
    mae = np.mean(MAE(forecast.loc[:date].values.squeeze(), real_values.loc[:date].values))
    smape = np.mean(sMAPE(...)) * 100
    print('{} - sMAPE: {:.2f}% | MAE: {:.3f}'.format(str(date)[:10], smape, mae))

    forecast.to_csv(forecast_file_path) # ★ 每轮都落盘, 中断也有部分结果
```

1.3 数据集

Epftoolbox 内置五个市场:

PJM(宾州-新泽西-马里兰), NP(Nord Pool 北欧), BE(EPEX 比利时), FR(EPEX 法国), DE(EPEX 德国)。另注: 库内置市场仅上述 5 个 (_datasets.py 白名单); 辽宁数据与天池数据不属于库内置, 是本仓库本地新增的转换产物 (examples/prepare_liaoning.py、prepare_tianchi.py)

1.4 评估指标

1.4.1 MAE:平均绝对误差

实现: `np.mean(np.abs(p_real-p_pred))`;

含义: 平均来说每次预测差多少钱。单位就是元/MWh, 所以很直观

1.4.2 sMAPE:对称平均绝对百分比误差

实现: `np.mean(np.abs(p_real-p_pred)/((np.abs(p_real)+np.abs(p_pred))/2))*100`

含义: 平均来说每次预测差多少比例

口径差异: epftoolbox 的 sMAPE 无下限; EFM3 的 sMAPE 有 floor50(真实值或者预测值 <50 时按 50 算)。二者数字不可直接比较

1.4.3 R²:决定系数

属于 EFM3, 该项目中没有

含义: 衡量"有没有抓住价格的波动"

$R^2=1$: 完全预测准确

$R^2=0$: 效果等同于"永远预测真实均值"

$R^2<0$: 比 $R^2=0$ 的效果还差

1.4.4 MSE:均方误差

几乎只出现在数学推导和损失函数里, 不做汇报

1.4.5 RMSE:均方根误差

实现: 就是给 MSE 开根号——目的是把单位还原成"元/MWh"

含义: 差的最狠的时候有多差; 平方差会放大误差, 这正是电价预测需要的(一次尖峰没抓住后果远大于十次小偏差)

1.4.6 RMSE/MAE (经验判据, 非项目实现)

比值: 1.0-1.2 说明误差均匀; >1.5 说明少数几个大误差在主导(典型是尖峰没抓住)。注: 该项不是 epftoolbox 实现的指标 (evaluation/ 下无对应函数), 此处仅作误差结构判读

1.5 结果

数据	模型	口径	结果
Nord Pool	LEAR	论文同窗口	sMAPE 4.88% (论文 4.85%)
PJM	LEAR	728 天	sMAPE 12.46%
PJM	DNN	64 天窗口	sMAPE 6.78% (同窗 LEAR 7.96%)
辽宁	LEAR	31 天	sMAPE 76.19% / MAE 100.03
天池	LEAR	2023-08~2024-04	sMAPE 73.93% / MAE 180.7

复现了论文: Nord Pool 上模型 4.88%, 论文上是 4.85%。注: 仓库内未见 NP 的复现产物 (examples/experimental_files/ 下只有 datLiaoPO / datLiao / datPJM / dattianchi 四份 LEAR 预测与一份 PJM DNN 预测), 此数字待补产物或注明未留存; 与论文对比时需写明测试窗口与校准窗口 (--calibration-window)

【留待思考】 如果该项目已经足够好, 那为什么不直接上线呢? 还要进行改进?

EFM3: 怎么投入生产

2.1 领域约定: 小时映射与分时段

即 efm3 把一天切成 24 小时。编号为 1...24

- 解决的问题: 没统一口径, 数据库, 模型, 评估会各自错位 1 小时—这种错误很隐蔽
- 怎么分:
- 小时映射: 00:00 映射成 24, 01:00 映射成 1...在数据库里的 target_date=2026-03-01, hour_business=24 就是 2026 年 3 月 1 号的 00:00

注: 数据库里实际存的是“当日 00:00”那一行; 而项目文档 (utils/business_day.py) 定义的 h24 是“次日 00:00”。两者相差一天, 属于待核实的口径不一致。

- 分时段: 不是把一整天当作一整块来预测, 而是把 24 小时划分为: 1-8 是谷, 9-16 是平, 17-24 是峰, 每一段训练一个模型, 预测时按小时三段和平段(9-16)负电价分类器拼接起来, 即谷回归+平回归+峰回归+平段负电价分类器。因此共有四个模型对象。

为什么要这么切：

谷段（00:00–08:00）：负荷最低，火电压到最小技术出力，新能源还在出力，价格常年贴近下限；

平段（09:00–16:00）：光伏出力冲高，净负荷被压出“鸭形曲线”的深谷，负电价集中出现在这里；

峰段（17:00–24:00）：光伏退坡、负荷攀升，是尖峰价格高发时段。

不这么切会怎么样：整体模型会迁就最复杂的时段，简单时段被迫一起变复杂，容易过拟合；另外，这三段用的不是同一套参数，这才是分段的意义。

时段	树数	学习率	叶子数	附加件
谷 1-8	2000	0.05	31	—
平 9-16	3000	0.03	63	负电价分类器
峰 17-24	3000	0.03	40	—

但代价是：

- 样本被切薄，每段只剩 1/3
- 08:00、16:00 是机制的切换点，硬切会产生人为断层（详见 2.3.9）
- 三倍的维护成本

注：分时段在项目里出现了两次——一次是生产电路在训练时切模型，一次是 fusion/ 包在融合层按时段分别学权重（后者详见 2.4.1）。

2.2 数据摄入：回填与幂等

把源文件(csv/xlsx)批量灌进 MySQL 的流程(data_ingestion)

核心思想：数据摄入的目的并不是为了把数据读进来，而是让每一行数据都清楚三个问题：

- 你来自哪个文件
- 你是哪一小时的
- 你这批数据在什么时间点前合法的

流水线：



【重要前提】 以下为通用框架的设计意图。生产链路实际使用的是

tools/db_ops/backfill_shandong_pmos_csv.py 这个一次性回填脚本——本框架的入口存在接口错误，从未成功运行。

1. 定位数据源：解析顺序是环境变量优先，取不到再用 YAML 里的 default_root；另外还约定了一条安全约定：数据库密码绝不能写进这个 YAML。只能走 --db-url 或者 EFM3_DB_URL

解决什么问题：数据目录写死在代码里，换台机器或者换个数据盘就得重新改动代码，而且哪些文件夹不该被扫描这个规则不清楚。

怎么解决：把“路径+白名单+黑名单”全部外置成一份 YAML，路径用环境变量覆盖，规则用 glob 表达，想加数据源改配置而不改代码。

去掉会怎么样：进入了“需要每个人本机改路径”，而且排除规则没了还会扫描 .venv，Node_modules 里的几万个文件，一起 hash，速度极慢。

2. 扫描文件：按照 Include_patterns 逐个 glob，命中后还要过一遍 exclude 过滤；每个文件算 sha256(64kb 流式读取，大文件不是一次性读入内存)；记录 path/name/ext/size_bytes/mtime/sha256 六元组；按照绝对路径去重。

另外有个值得学习的地方：碰到 OSError 或者权限错误时只记录下 warning 并跳过该文件，不中断扫描，这样目录里那些可能经常被占用的 excel 打不开时不该让整批数据更新失败
为什么要算哈希，解决什么问题：需要判断“这个文件跟上次比有没有发生变化”，文件名和修改时间不可靠

怎么解决：用文件内容的 hash 值判断，内容一样值就一样

去掉后会怎么样：只靠修改时间(mtime)和文件名，很可能会漏更新，白跑(内容没变却重导入一次)，而且审计时无法证明这批数据用的是哪一版

3. 登记与增量判断：“省时间”装置，有四个状态：NEW / SKIPPED / CHANGED / FAILED

状态	触发条件	后果
NEW	库里查不到这条记录	导入
SKIPPED	sha256 与库里一致	跳过，不重复导入
CHANGED	同名但内容变了	重新导入
FAILED	查询或写入异常	记录错误

然后 UPSERT 进 efm_source_files 表

解决什么问题：数据每天都会发生变化，如果每次都重新导入文件越攒越多导致系统越来越慢；全量重导还可能让“昨天的结果”和“今天的结果”不一致，指标无法复现。

怎么解决：把文件是否变过变成一次数据库查询，只有 new 或者 changed 才导入

去掉会怎么样：要么就是重跑这些文件，导致系统越来越慢；要么人工标记哪些导入过了，并且也无法确定这批数据到底来自文件哪一版

4. 解析入库：

- 认格式：xlsx 用 openpyxl, xls 用 xlrd, csv 先试 GBK 不行再用 UTF-8
- 认列：靠一张写死的映射表把中文列名映射成英文指标，共有 22 列
- 规范时间：时间列叫“时刻”，解析失败的行直接丢弃；然后做国内现货最关键的换算：00:00 → 24，绝不允许出现 0
- 宽表转长表：最巧妙的一步，把列名变成一行数据

为什么用长表，解决什么问题：不同来源的列名，列数都不一样，用宽表意味着每接一个新来源就要更改表结构；而且这个指标哪来的什么时候更新的也不清楚。

怎么解决：把指标从表结构降级为一条普通数据，加指标等于加行，不用改表

去掉后会怎么样：回到宽表，字段一多出现上百列，大量 NULL，改一次表结构要动大量代码。

举个例子：

【宽表】一行 = 一个学生，科目横着排				
学号	姓名	语文	数学	英语
S01	张三	90	85	88
S02	李四	78	92	81

【长表】一行 = 一个学生的一门成绩，科目竖着排		
学号	科目	分数
S01	语文	90
S01	数学	85
S01	英语	88
S02	语文	78
S02	数学	92
S02	英语	81

行数：宽表 2 行 → 长表 6 行（学生数 × 科目数）。

5. 幂等写入：所有 INSERT 都带 ON DUPLICATE KEY UPDATE，这意味着同一份数据重新导入多次库里的结果完全一样——可以放心重跑，方便复现；我能够重跑两遍得到一致结果的原因

6. 质检：对每个文件做六件事：

分别是：24 行/日、小时范围[1,24]、重复小时、缺失小时、价格区间[-500, 5000]、D14 合规

检查项	规则	为什么这条重要
row_count_24_per_date	每个(日期, 指标)恰好 24 行	少一行模型不报错，只会悄悄变差
canonical_hour_mapping	hour ∈ [1,24]，出现 0 就是错	0 和 24 混淆会把“次日零点”错算成“当天第一点”
duplicate_hours	无重复 (市场, 指标, 日期, 小时)	重复行会让同一天被算两次
missing_hours	每天每指标 24 个不同小时都齐	同上
price_range	电价 ∈ [-500, 5000] 元/MWh	抓单位错误（元 vs 元/MWh 差 1000 倍）和脏数据
d14_cutoff_compliance	最大交易日必须早于“今天”	防未来数据泄露

为什么质检要单独一层，解决什么问题：数据换行，单位错，小时错位这些问题模型不会报错，只会学出一堆垃圾，表现为模型不准

怎么解决：把电力行业的常识翻译成机器能执行的断言，在数据刚入库时就审查，而不是训练完才查、

去掉会怎么样：错误会一直存在，最后看到与真实市场的误差特别大，但却排查不出原因

7. 数据集版本：

```
1. 检查必需指标是否齐全；
2. 算出 D14 截止时间：leakage_cutoff = 目标日 - 1 天 14:00 (📌 第 160 行 (line 160))。例：预测 2026-03-11，则只能用 03-10 14:00 之前可得的信息；
3. 生成 dataset_id = ds_2026-03-11_shandong_a1b2c3d4；
4. 把这批数据涉及的所有源文件哈希、各指标行数一起记进 efm_dataset_versions；
5. 状态判定：缺必需指标 → FAIL；行数不足 → PARTIAL；全齐 → READY。
```

解决什么问题：不知道该怎么证明有没有用到未来数据

怎么解决：把数据集变成一个带 ID，文件哈希，截止时间的可追溯对象

注：具体规则：leakage_cutoff = 目标日 - 1 天 14:00（代码中称为 D14）。

去掉会怎么样：结果不可复现，无法保证到底用没有未来数据

2.3 台账与血缘

2.3.1 台账与血缘是什么

把预测和实际都像账本一样记录，每条记录带有完整的身份信息

解决什么问题：回答—这一天的预测是谁，用什么模型，基于什么数据，什么时候算的

怎么解决：每条预测记录都带有这些字段：

字段	含义
run_id	哪一次运行
model_name / model_version	哪个模型、哪个版本
data_cutoff	用了截止到什么时候的数据（防泄露的证据）
forecast_date / target_day / hour_business	何时预测、预测哪一天、第几小时
created_at	什么时候写入

去掉会怎么样：模型上线后无法复盘，无法审计，无法回答为什么那天预测错了

	台账 Ledger	血缘 Lineage
回答的问题	“发生了什么”	“为什么是这个数”
形态	一行行记录，只增不改	节点 + 边构成的图
存放位置	23 张 MySQL 表	读取时现算，不落库
类比	银行流水	资金流向图

2.3.2 为什么要做台账和血缘

解决什么问题：

- 复盘无据：比如模型性能突然变差，我想知道哪一步导致变差，如果只有一份最终的 csv 文件，只能看到结果看不到中间发生了什么
- 无法自证：无法证明到底用没有未来数据
- 影子模型污染：EFM3 里同时跑正式模型和一堆实验模型。一旦某个实验模型被选入正式交付，就是生产事故。这种事必须由系统自动拦截。

怎么解决的：

- 每一行标记 run_id 和 stage，谁生产的和扮演什么角色
- 关键关系用外键和边表固化，谁来自谁
- 用检查项把纪律变成断言
- 去掉后会怎么样：系统退化成一堆 csv 文件和一份 markdown 报告。该项目取代了“指标只在 markdown 里”的做法，这个做法便是 epftoolbox 的状态：结果全在 experimental_files 里的一堆 csv 文件，想要知道里面某个结果到底怎么来的就必须重新跑一次。

2.3.3 23 张表分五组

组	表	职责
A 运行层 (3)	efm_runs / efm_run_events / efm_pipeline_steps	这次运行是什么、跑到哪、每步耗时多少
B 数据层 (6)	efm_data_sources / efm_source_files / efm_data_update_runs / efm_market_data_hourly / efm_actual_prices / efm_dataset_versions	数据从哪来、是哪一版（上次课讲的全部内容）
C 特征与预测 (6)	efm_feature_snapshots / efm_predictions / efm_prediction_batches / efm_fusion_candidates / efm_repair_decisions / efm_task_finals	每个模型每个阶段的输出
D 决策与交付 (4)	efm_fusion_decisions / efm_delivery_finals / efm_delivery_outputs / efm_postflight_checks	怎么选的、交了什么、检查过了没有
E 血缘与治理 (4)	efm_prediction_lineage_edges / efm_metric_runs / efm_model_registry / efm_artifacts	边、指标、模型登记、产物

数据库共 23 张表：db/schema.sql 定义 15 张，db/migrations/001-005 去重后共 23 张（新增 8 张：efm_pipeline_steps、efm_prediction_batches、efm_prediction_lineage_edges、efm_repair_decisions、efm_fusion_candidates、efm_task_finals、efm_delivery_finals、efm_metric_runs），里面还有五条纪律：

永不 DROP 或 ALTER 已有表；每个 CREATE 都带 IF NOT EXISTS；MySQL 8 兼容；热点查询路径都建了索引；不删数据，天然幂等。

这便是生产级和课程作业的区别：迁移脚本可以反复执行，且不会破坏已有数据

2.3.4 最核心的一张表：efm_predictions

它是所有预测的总帐本：

```
run_id      -- 哪一次运行
target_date -- 预测哪一天
hour_business -- 哪个小时 (1..24)
task        -- dayahead | realtime | fusion | final | shadow | delivery
stage       -- raw_model | da_anchor | official_baseline | selector_shadow
            -- | p3_shadow | seasonal_da_router | final_selected ...
model_name
pred_price
is_shadow    -- 是不是实验模型
is_selected  -- 是不是最终被选中的那个
selected_reason -- 为什么选它
cutoff_time  -- 这个预测做出来的时间点
quality_flags
UNIQUE KEY uk_run_date_hour_stage (run_id, target_date, hour_business, stage, model_name)
```

设计精髓在于：is_shadow 和 is_selected

is_shadow 回答：这个模型有没有资格上生产——即便没有资格进入生产也会入库但会打上 shadow 标签，确保不会进生产

is_selected 回答：这一个小时最终用了谁——24 小时里每个小时有且只有一行是

is_selected=1，但数据库并未强制要求这一点，是靠 postflight 的 selected_source 检查来保证的。

为什么要这么设计：

- 解决什么问题：如果实验模型不允许入库，那就无法做“如果用了实验模型结果会怎么样”的回测；如果允许入库但不打标记，某天可能会进入到生产里。
- 怎么解决：允许入库但是强制打标。
- 去掉会怎么样：要么不让实验模型跑；要么允许跑但是分不清楚是谁，迟早会污染生产。

2.3.5 血缘怎么连起来：三种机制

不靠一张表，而是靠三层连接：

- 外键直连：表与表之间用真实外键连接

子表.字段	指向	含义
efm_fusion_decisions.selected_prediction_id	efm_predictions.id	这次路由选了哪一行预测
efm_fusion_candidates.candidate_prediction_id	efm_predictions.id	这个候选是哪一行
efm_task_finals.final_prediction_id	efm_predictions.id	日前/实时最终值来自哪一行
efm_delivery_finals.dayahead_final_id / realtime_final_id	efm_task_finals.id	交付价由哪两个 final 合成
efm_prediction_lineage_edges.parent/child_prediction_id	efm_predictions.id	一条预测变成另一条预测

有了外键，数据库层面就不可能存在一个"指向不存在的预测"的记录，并且删除 run 时 ON DELETE CASCADE 会自动清理干净

- 边表(图结构)

Migration 表中：

```
parent_prediction_id, child_prediction_id,
relation_type ENUM('repair','weight','fuse','select','fallback',
'classifier_adjust','separator_adjust','negative_fix'),
relation_json
```

枚举把"一次价格变化的原因"分为了八类：这八类是 EFM3 里"价格被改动"的全部可能路径。任何一个最终价往上追一定能追到这几类

关系类型	含义
repair	模块修复（缺值/异常被修正）
weight	加权组合
fuse	融合
select	被选中为最终值
fallback	降级回退（模型没出结果，退到备用）
classifier_adjust	分类器调整（还记得平段那个负电价分类器吗）
separator_adjust	分离器调整（日前/实时之间的切换）
negative_fix	负电价修正

- 业务键软连(三件套)

所有表共享：run_id+target_date+hour_business 这个三件套，是对齐尺——正因为每张表用同一个尺子，系统才能按"某天某小时"把整条链路拉出来；这也解释了为什么之前领域约定中把 24 小时统一编号：必须是 1~24 且 00:00 必须是 24，只要有一张表用了 0 表示零点，整条血缘就断在了这一小时上。

2.3.6 一次运行往台账里写了什么（时序）

阶段	写入的表	记了什么
启动	efm_runs	run_id、目标日、git_sha、config_hash、模式、状态
每一步	efm_pipeline_steps	步骤名、顺序、状态、输入行数/输出行数、耗时 ms
过程中	efm_run_events	start / step / warning / error / complete
数据准备	efm_dataset_versions	数据集 ID、文件哈希、D14 截止时间
特征	efm_feature_snapshots	特征 JSON + feature_hash
各模型出价	efm_predictions + efm_prediction_batches	每个 stage 一行，带批次归属
修复	efm_repair_decisions	before_value / after_value / rule_name / severity
融合	efm_fusion_candidates	每个候选的 weight_value、rank_value、selected、rejected_reason
路由决策	efm_fusion_decisions	policy_name、selected_model、decision_json
日前/实时最终	efm_task_finals	两个任务分开存，永不混成一行
交付	efm_delivery_finals	交付价 + dayahead_final_id、realtime_final_id、separator_rule、fallback_reason
导出	efm_delivery_outputs / efm_artifacts	文件路径 + file_hash
校验	efm_postflight_checks	8 项检查各自 passed / details
指标	efm_metric_runs	sMAPE / MAE / RMSE / MAPE / WMAPE + metric_scope

三处设计要点：

- efm_runs 里的 git_sha 和 config_hash：
光记录什么时候跑的还不够，还要记录“用的是哪一版代码和哪一份配置”，否则很难复现
- efm_task_finals 把日前和实时分开存储：
业务上的要求，日前价和实时价是两个不同的交易指标，混在一列迟早出事
- efm_repair_decisions 连什么都没有改时也要记录：
很关键，如果只记改过的，那无法区分“这个小时没触发规则”和“触发了但规则判断不用改”。
记录没发生什么和发生了什么同样重要

2.3.7 血缘图怎么算出来的：不入库，读时现算

为什么不用图数据库：

解决什么问题：

血缘天然图，直觉上该用图数据库

怎么解决：

选择不引入，而是把关系存成普通表(外表+外键)，查询时用 SQL 拼出节点和边

去掉会怎么样：

如果上真的图数据库，为了一个读一次画一次图的场景引入一套新数据库是典型的技术过度设计

2.3.8 期望改进

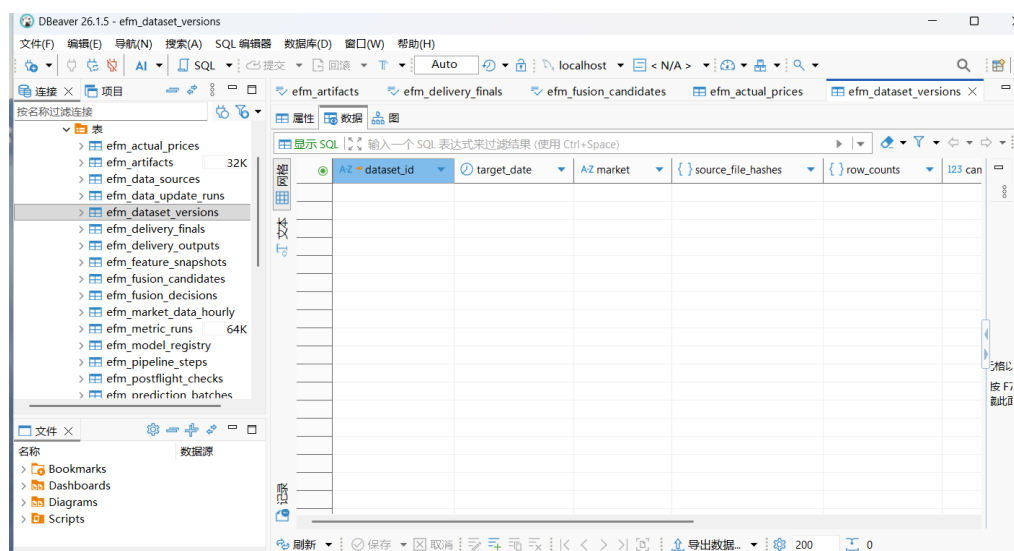
- 血缘图是现算的，一次查看血缘 (get_lineage)要发十条查询 (lineage_service.py:92-162: run / dataset / source_files / feature / candidates / shadows / router / selected / postflight / delivery)，单小时查没问题，但想一次性看 98 天*24 小时的血缘，需要查上万次。可以加上缓存
- Efm_dataset_versions 按 target_date 后取最新一条：lineage_service 里：

```
WHERE target_date=%s ORDER BY created_at DESC LIMIT 1
```

如果同一天跑了多次数据构建，血缘挂到的是最后一次构建的版本，而当时那次 run 实际用的可能是更早的那版。但这张表本身就是空的，efm_runs 里也没有 dataset_id 这一列，所以运行和数据版本之间根本没有绑定关系，没有版本可指，但模型不会报错；

另外 efm_feature_snapshots, efm_delivery_outputs, efm_model_registry 也全都是空表。到底发生了什么？

- 数据摄入没做？但数据在库里，所以不是这个问题
- 数据摄入有缺陷？代码里确实有缺陷（调用的方法不存在、CHANGED 状态不可达、dataset_id 不是内容指纹），但因为模块从未运行，这些缺陷没有触发。“有缺陷”和“缺陷发作了”要分开说
- 项目明显设计了完整的通用数据摄入框架，但生产链路实际却绕开了它，改用一次性回填脚本；通用框架的调用入口存在接口错误未成功执行，因此数据集版本和特征快照两张审计表都为空：



血缘链路上游断开：数据版本和特征快照两环只有代码没有数据。

表	行数	状态
efm_runs	203	✅ 98 天的运行记录都在
efm_predictions	17,904	✅ 所有模型的出价
efm_prediction_lineage_edges	384	✅ 血缘边
efm_postflight_checks	2,326	✅ 交付校验
efm_run_events	1,800	✅ 事件日志
efm_market_data_hourly	4,704	✅ 市场数据
efm_actual_prices	2,352	✅ 实际价格
efm_metric_runs	3	✅ 指标
efm_dataset_versions	0	❌ 空
efm_feature_snapshots	0	❌ 也空
efm_delivery_outputs	0	❌ 空
efm_model_registry	0	❌ 空

2.3.9 分时段硬切可能导致人为断层

断层指的并不是价格的跳变，而是模型行为的突变。

三时段的接缝有两个，分别是：

接缝一：h8（07:00–08:00）用谷模型，h9（08:00–09:00）用平模型

接缝二：h16（15:00–16:00）用平模型，h17（16:00–17:00）用峰模型

怎么产生的：

三个模型是三个互不相干的函数；谷模型只在夜间数据上训练，它根本没见过 9 点的价格；平模型只在白天的数据上训练，不知道凌晨的价位；他们分别在自己的样本上追求最优，没有任何机制保证两者在接缝处衔接得上。

更具体而言：三个模型都有各自学出来的基准，谷模型学到的夜间价位偏低，那它对 h8 的预测就容易偏低；而 h9 换成一个不知道夜间水平的模型来预测，于是 08:00 那个点的价格更多的是反映了“现在轮到哪个模型起到了主导作用”，而不是价格的锅：

这是 2026-03-01 的真实日前价的相邻小时变化量：

位置	变化	是不是接缝
h7 → h8	+97.1	✗ 不是
h8 → h9	+25.0	✓ 是接缝
h12 → h13	-50.3	✗ 不是
h16 → h17	-40.3	✓ 是接缝
h23 → h24	+49.8	✗ 不是

可以看到，这一天相邻小时的平均变化是 23 元/MWh，最大的一次跳变(+97)发生在 7-8 点，而这个时间点根本不在接缝上。这说明：真实电价本来就会在任何地方跳

怎么解决：边界小时共享、重叠训练、后处理约束、加时段特征

2.4 日前链路：三模型融合 + 权重学习

日前价格用 LightGBM+XGBoost+CatBoost 三个模型并行预测，再融合成一个结果

解决什么问题：由于单模型可能会有偏好和方差，集成模型可以降低某一个模型在某种行情下崩掉的风险。

怎么解决：

1.权重学习：

选最近 N 天“完整日”(默认窗口 30 天，最多回溯 90 天)。

“完整”的定义很严格，24 小时齐全，预测和实际都不缺，无 NaN，小时集合匹配等。

输出按(task*period)的权重，即“日前/实时*谷/平/峰”各自一套权重。

落盘：weights.csv 与 dynamic_weight_trace.csv（权重随时间的变化轨迹）

2.融合：apply_daily_ledger_weights.py

按 task+period 取权重->只保留当天真正有预测的模型->重新归一化->逐小时加权平均。

严格模式：缺小时/模型/权重就报错，只有 allow_equal_weight_fallback 打开时才退化为等权。

去掉会怎么样：三个模型独立，无法合成一个交付值；或者用简单的平均，对表现差的模型一视同仁，白白浪费好的模型。

【阅读提示】 本节先讲 fusion/ 包的“权重学习”，再讲生产电路的“五阶段”。两者是两套不同的实现，区别见 2.4.6。

步	stage 名	代码	做什么	输出行数
3	dayahead_raw_model	model_loader.py (line 49)	从库里读各模型原始预测	24 × 模型数
4	dayahead_module_repaired	repair_chain.py (line 56)	守卫规则修正异常值	24 × 模型数
5	dayahead_fused	fusion_chain.py (line 57)	多模型加权融合	24
6	dayahead_classifier_adjusted	classifier_chain.py (line 46)	日前是透传	24
7	dayahead_task_final	dayahead_chain.py (line 154)	定稿，写权威表	24

逐阶段详解：

2.4.1 阶段一：取模型原始输出

设计的关键是：按 target_date 读，不按 run_id 读；原始数据是由外部进程产出的，每个交易日各生成一次，不属于某次电路运行。

默认模型是 cfg05 / xgboost_rich / catboost_rich，可以用 config["dayahead_models"] 覆盖（我那次实际用的是 lightgbm / timemixer / timesfm）。

这一阶段最重要的一条纪律是：诚实状态

- 解决什么问题：da_anchor(日前实际出清价)是一份免费的基准，如果模型还未出结果就用它，会得到准确率特别高的结果，但这并不是模型，是用日前价预测日前价。
- 怎么解决：默认禁止兜底。要使用时必须显示打开 allow_benchmark_fallback=True，而且写进去的 stage 叫 benchmark_da_anchor，model_name 叫 da_anchor_benchmark，quality_flags 标 benchmark，这样从字段方面就冒充不了模型。
- 去掉会怎么样：基准默默成为模型，指标失真；预测结果可能很好看，但实际的预测能力很垃圾。

2.4.2 阶段二：修复（三条守卫规则）

规则	触发条件	处理
no_nan	值是 NaN	用全天中位数填
range_guard	超出 [-500, 2000]	截断到边界
spike_guard	偏离中位数超过 3σ	拉回中位数

值得记的设计细节：

- 无论改没改，都记一条决策。Insert_repair_decision 在循环里无条件执行，no_op 也记。
- 解决什么问题：还是那个问题，如果只记改过的，就分不清“触发但判断不用改”和“没触发规则”

去掉会怎么样：复盘时某小时没有修复记录，不知道是正常情况还是漏了

- 修复不改变模型的身份。Model_name/model_version 原样带下去；按照作者说明：如果修复时把三个模型揉成一个，融合就没法按模型加权了；另外每一条 raw->repaired 还会写一条血缘边，没有输入时->skipped 绝不编造。

2.4.3 阶段三：融合：fusion_chain.py

每个候选都单独写一行 efm_fusion_candidates，带 weight_value 和 score_json（含 mode / n_candidates / raw_weight）。所以融合过程是完全可审计的，不仅知道结果，还知道每个模型占了多少份量。

但我那次实际跑出来的是等权：三个模型各占一份，是因为 config 里没传 fusion_weights，全都默认 1.0 再归一化。

2.4.4 阶段四：分类器（日前是透传）

按照作者意思：分类器是实时链路专有的(像之前的负电价分类器那类东西)。日前保留这一步是为了审计的连续性，让 stage 序列和实时链路结构对称，前端和血缘图不用写两套逻辑。

而实时那一侧的分类器目前是 PLACEHOLDER；照样写 classifier_adjusted 行(保证血缘不断)，但步骤状态记 SKIPPED，不记 COMPLETE。

- 解决什么问题：功能没迁移时如果直接不写这一步血缘图会缺一环；如果写成 COMPLETE，别人会以为分类器在工作。
- 怎么解决：写 数据+标状态。数据照写(审计用)，状态如实标(SKIPPED)
- 去掉会怎么样：要么血缘断开，要么把没做好的功能伪装做好了

2.4.5 阶段五：定稿

优先读 classifier_adjusted，读不到就退到 fused，然后同时写两处：

- efm_predictions 的 stage = dayahead_task_final
- efm_task_finals（日前最终表）

并写一条 relation_type = 'select' 的血缘边。日前和实时的 final 分表分区存放，永不混行。

2.4.6 关键补充：仓库里存在两套融合实现

	fusion/ 那套	production_circuit/ 那套
权重来源	学出来的 (SLSQP 最小化 smape_floor50)	配置里传的 (缺省 1.0 → 等权)
权重粒度	(task × period), 三个时段各一套	全局一套
输出	CSV: <code>outputs/runs/{D}/{task}/fuse/</code>	数据库: <code>efm_predictions</code> + <code>efm_fusion_candidates</code>
入口	<code>python main.py --pipeline ledger_fuse</code>	<code>run_production_circuit()</code> (DAG 第 5 步)

同一天学出来的权重（见下图）：

时段	lightgbm	timemixer	timesfm
1_8	0.108	0.065	0.826
9_16	0.049	0.049	0.902
17_24	0.231	0.098	0.671

融合存在两套实现，口径未统一：

- fusion/apply_daily_ledger_weights.py——学权重融合，按 (task×period) 学到权重（TimesFM 权重可达 0.9），结果落盘 CSV，未回写数据库；
- production_circuit/fusion_chain.py——配权重融合，权重来自配置（缺省等权 1/3），结果写入 efm_predictions / efm_fusion_candidates。

数据库、血缘、efm_task_finals 引用的是第二套；两者在 2026-02-24 的融合结果平均相差 29.87 元/MWh。

2.5 实时链路：选择器与回退

实时价差任务更难，EFM3 不是全都上，而是按季节/可用性挑一个来源。

解决什么问题：实时链路候选很多(SGDFNet / TimesFM / 日前锚)，盲目融合性能反而更差；而且数据不全时必须能降级。

怎么解决：

文档描述的三级回退：官方基线 → SGDFNet → 日前锚。但实测行为与文档描述不同——见下方实测结果。

```

is_winter = _is_winter(target_date)
if is_winter:
    raw_preds = ... read(stage="da_anchor") # 冬季：直接日前锚
else:
    raw_preds = ... read(stage="official_baseline") # 非冬季优先官方基线
    if not raw_preds:
        raw_preds = sgdfnet 的原始模型 # 退一步：SGDFNet
    if not raw_preds:
        raw_preds = ... read(stage="da_anchor") # 最后退到日前清价

```

去掉会怎么样：没有这个选择器，所以模型必须都存在时才能出结果；缺一个模型整条链路就中断，有了它缺数据也能出结果，只是精度较差。

我这次跑出的结果: 32.66% / 101.98 就是退到第三级日前锚的结果，这只是基准精度而不是三个模型融合的精度；实时来源由 seasonal_da_router 按照季节选择：

- 冬季直接取日前锚
- 非冬季按三级后退

但实测 4800/4800 条决策全部落在 da_anchor——32 天的实时模型输出全都在冬季，被直接绕过；3-4 月非冬季又没有模型输出；因此实际上等价于只用“日前锚”

另外：生产电路侧一套 da_aware_sgdf_selector，策略则相反，日前锚为主，SGDFNet 为辅。

2.6 极值处理与防泄漏

- 专门应对负电价和尖峰，
- 用协议保证只用当时可知的信息；

解决什么问题：

- 山东/甘肃等地 $|price| > 500$ 的尖峰和负电价频繁，普通回归易被带偏，
- 生产系统最怕的就是未来泄露，离线指标很完美，一上线就暴雷

怎么解决：

- 极值：ExtremPriceClf / 极值价格分类器（先判断这一小时是否极端，再单独处理）+ 负电价修正模块。另外 TimeMixer 管线提供 VMD（变分模态分解）作为可选开关，但默认关闭，SGDFNet / RT916 并未使用模态分解。
- 防泄漏：cutoff-safe 协议——实时链路的数据截至 D-1 的某个时点（参数 --realtime-cutoff-hour，默认 14，即 D-1 日 14:00，代码中称为 D14）。注意 docs/ 下两份 p3_spike_residual 文档（docs/p3_spike_residual_existing_experience.md:34、docs/p3_spike_residual_final_report.md:21）写的是 D15，与代码 D14 不一致（README.md 全文无 D15），建议统一写 D14。

去掉会怎么样：

- 去掉了极值处理：尖峰/负电价的预测误差大的离谱，指标被少数时段拖垮

- 去掉 cutoff-safe 协议：离线指标虚高(未来泄露)，上线后真实表现大打折扣

注：判断某个特征能不能用不是看数据里有没有，而是看预测那一刻是否已经知道。

2.7 编排、状态机与交付校验

一个编排器把 19 个步骤串了起来，并决定“这次运行到底算成功还是失败”

解决什么问题：生产系统不是跑完就完事了，而是要回答 交付到底能不能发

怎么解决：

```
steps["dayahead_prediction"] = ...
steps["realtime_prediction"] = ...
steps["candidate_shadow_prediction"] = ...
steps["seasonal_da_router"] = ...
steps["final_selection"] = ...
steps["postflight"] = ...
steps["export"] = ...
steps["formal_guard"] = ...
steps["update_run_status"] = ...

状态机核心 ( _determine_delivery_status ):

</> Python

critical_steps = ["dayahead_prediction", "realtime_prediction",
                  "seasonal_da_router", "final_selection"]
if mode in ("formal", "formal_sim"): critical_steps.append("formal_guard")

if not critical_ok: return 2, "FAILED_NO_DELIVERY" # 关键步骤挂了 → 不交付
if all_ok: return 0, "NORMAL" # 全部通过 → 正常交付
if any_failed: return 1, "DEGRADED_DELIVERED" # 非关键步骤挂了 → 降级交付
```

配套的两个机制：

- 影子模式和正式模式：shadow 只演练不交付；formal 才写 submission_ready.csv，formal_sim 走严格护栏但不导出，我们跑的就是 formal_sim；

影子隔离的三层防护：is_shadow 打标；shadow_not_final 检查：血缘图的 shadow_safe 徽章

- Postflight 8 项校验：交付前自检，8 项都通过才允许交付。

8 项 postflight 的名称 (pipelines/db_postflight.py:20-29 的 _CHECK_NAMES)：

row_count_24、hour_range、no_nan、no_duplicates、price_range、selected_source、shadow_not_final、submission_row_count

去掉会怎么样：

没有状态机就无法区分“跑完了”和“跑对了”；没有 Postflight，错误数据就会被直接送到对方手里。

2.8 指标与审计口径

把台账里的预测和实际，按不同的口径算成指标

解决什么问题：

同一份数据，“用哪个口径算”会得出完全不同的数字——口径说不清。汇报必然被质疑

怎么解决：用 metric_scope 把口径写进数据库枚举——dayahead / realtime / delivery / benchmark 四种，作者原话：benchmark 永远不会被当作模型指标报出去。

scope	含义
benchmark	da_anchor vs rt_actual（日前价当实时价的预测）——我们跑的就是这个
dayahead	日前模型 vs 日前实际（需真有模型 final）
realtime	实时模型 vs 实时实际
delivery	交付值 vs 实时实际

使用山东数据的结果(共 98 天)：

98/98 天 COMPLETE、0 FAIL、Postflight 8/8、formal guards PASS；

benchmark：SMAPE 32.66% (基准指标)/ MAE 101.98(元/MWh)/ RMSE 136.76；

分月 MAE（元/MWh）：1 月 102.97；2 月 111.78；3 月 88.75；4 月（1-8 日）115.14。

去掉口径区分会怎么样：把“基准精度”当成“模型精度”

FutureBoosting：零样本预测特征 + 预测电价

3.1 项目目标

一句话：FutureBoosting 是一个两阶段混合预测框架——先用冻结的时序基础模型(TSFM)对目标交易日的驱动变量做零样本预测，再把这些预测当作特征喂给轻量回归模型(LightGBM/Ridge)预测电价曲线。是论文的官方实现，不是服务。

3.1.1 时序基础模型

1.解决什么问题：

传统的时序预测(epftoolbox 里的 LEAR 和 DNN)的方法是：一个市场，一套数据，训练一个模型，LEAR 甚至一个小时训练一个模型。需要每天重校验。这做法有三个痛点：

a. 还数据集就要重来

- b. 样本数量可能不够
- c. 点预测只能给一个数，无法预测未来有几种可能

TSFM 很直接：在海量，异构的公开时序数据上训练一个通用模型，推理时完全冻结权重，仅靠输入的历史窗口直接给出未来预测，也就是零样本。此外，TSFM 还可以把签发时刻拿不到的未来外生变量外推出来。

- 该项目的用法很聪明：TSFM 只负责造特征，最终决策交给可解释的回归模型

2. 怎么解决：(通用机制，与具体模型无关)

- 分块：把长序列切成固定长度的 patch，每个 patch 变成 transformer 的一个 token。这一步是把“数值序列”变成“模型能吃的序列”的关键。
- 大规模与训练：模型学到的不是“电价规律”，而是时序的通用形态(趋势，周期，突变，噪声)
- 概率输出：主流 TSFM 输出的是分位数而不是单点。因为预测未来的本质是不确定的，分位数能够同时给出“最可能值”和“可能范围”。
- 零样本推理：给定 context，直接输出 horizon 长度的一组样本或分位数，不更新任何权重。

与传统回归的本质区别：时间信息是怎么进入模型的？靠“位置”本身——序列的顺序就是时间轴，patch 的位置编码加上上下文长度，决定了模型能够看多远；不需要手工构造滞后特征，日历特征或滚动统计量。

3. 去掉会怎么样：

从“基础型增强的混合预测”退化为“纯滞后特征回归基线”。

3.1.2 真实存在的问题

在电力现货日前环节，预测是在 D-1 的某个时刻签发的。那一刻，次日 96 点(山西)或 24 点(欧洲)的系统负荷，新能源出力，火电竞价空间，联络线计划这些值并不存在真实值——要么只是预测值，要么根本拿不到。而一个纯回归模型只能吃“签发时刻已经存在”的滞后特征，也就是历史价格滞后项，历史天气，历史负荷。于是它天然缺了一整层信息：对未来供需形势的预期。而电价恰恰主要由供需决定，这层信息缺失，模型就只能靠“昨天同一时刻多少钱”这类模型外推。

怎么解决：

把这层信息用模型造出来，让 TSFM 从历史序列外推目标日的这些变量，把预测结果作为新的特征喂给回归模型。

去掉会怎么样：

通过 `-enable_tsfm 0`(只有 CPU，没有 TSFM 权重，能不能跑)。这时第二阶段的 TSFM 特征全线消失；但程序照样可运行，不报错，指标也能出来，但项目性质发生变化：从基础型增强的混合预测退化成纯滞后特征回归基线。

3.1.3 设计细节

山西日前配置里，被 TSFM 预测的变量表：
configs/columns/shanxi/dayahead/tsfm_ic94.json

有 94 项，它的最后一项就是回归目标本身：

day_ahead_clearing_price;

而有一条代码会：目标不在 TSFM 变量表里时强制把目标追加进去。

这意味着：TSFM 不仅预测负荷和新能源，也对电价本身做了一次预测，这个价格预测又作为特征进入回归。所以第二阶段的本质里包含一层 stacking/集成(基础模型的电价预测+回归对全部特征的重加权)，而不只是“给回归补几个外生变量”。

那它算不算未来泄露？

只用了锚点之前的历史，未来协变量根本没喂进去，所以这是“用历史外推出来的预测值”，不是未来真实值；因此不构成直接的未未来泄露，但它是该项目最需要进行泄露审查的一处

3.1.4 程序入口

整个项目只有一个入口：run_pipeline.py，参数分为四组：

- 数据与切分
- TSFM 开关与权重路径
- 回归器选择
- LightGBM 超参与 SHAP

3.2 数据入口与数据格式

本框架里，特征的语义是：目标时刻 t 的值，而非：签发时刻的值

这句话决定了整个方法论的成立与否——因为真实生产里，D-1 签发时根本不知道 t 时刻的负荷和价格。

3.2.1 原理一：变量必须按“签发时刻的可得性”分三类

1.解决什么问题：

不分类，就没法判断一个变量能不能进特征表.这是电价预测最容易忽略，后果最严重的一步：数据集里的所有列看起来都能用是因为它们都已经填好了。

2. 怎么解决：

按“这个值在 D-1 签发那一刻是否真实存在”分为三类：

类别	含义	例子	能不能做特征
A 类：历史已知	签发时刻就已经发生	昨天同时刻电价、前天负荷、星期几、是否节假日	能，直接做特征
B 类：未来已知（有预测版本发布）	目标是未来，但签发前会有官方/商业预测发布	系统负荷预测、新能源出力预测、数值天气预报	能，但必须用“当时发布的那个版本
C 类：未来才有的实测	只有目标日结束后才知道	实际负荷、实际风光出力、实际结算价、再分析气象	不能，用了就是泄露

关键在于：同一物理量可以属于不同类别。“负荷”在 A 类(昨天的实际负荷)和 C 类(今天的实际负荷)里都存在，而“负荷预测”是 B 类。这三者可用性完全不同但列名可能只差几个字。

3. 去掉会怎么样：

- 把 C 类当特征用：回测指标看着很正常但在生产里无法复现。整套项目全都白做
- 反过来，把 B 类误判成 C 类而排除：白白扔掉了最有价值的未来信息；在新能源占比高的省份，负荷预测往往是电价最强的单一解释变量，扔掉它模型就只能自相关。

3.2.2 原理二：框架不造特征，只认“已经算好的列”；TSFM 是唯一的例外

1. 解决什么问题：

很多人读这类框架时默认“它会自动生成滞后特征，日历特征”。实际不是，这个框架时列选择器+外推器，不做特征构造。

2. 怎么解决：

- 两份 JSON 配置分工明确：regress_cols 是“回归能吃的全部列”(最后一列是目标)，tsfm_cols 是“需要被外推的列”。两者可以不同，因为责任不同——前者是最终喂给回归的信息集，后者是签发时刻缺哪些未来值。
- 唯一的例外是 TSFM 阶段：是框架里唯一真的在造新特征的环节——把 C 类变量外推成预测值，从而把他们降级成可用的 B 类

3. 去掉会怎么样：

- 忽略“列必须预先算好”这一点：会一直在配置里写不存在的列名或者错把原始 price 当成滞后项用。
- 关掉 TSFM：特征表里 pred_开头的列全部消失。项目从“基础型增强的混合预测”退化“成”纯滞后特征的回归基线”

3.2.3 原理三：粒度决定滞后步长也决定模型能看到什么周期

1. 解决什么问题：

“昨天同一时刻”在 15 分钟数据里就是滞后 96 步，在 24 小时数据里就是滞后 24 步；粒度不同，同一个物理含义对应完全不同的数字；写错一位模型学到的就是另一个东西。

2. 怎么解决：

- 粒度，日周期长度的映射：15 分钟=96 点，小时=24 点
- 粒度越细越关注：
 - a. 日内形态：谷/平/峰的结构在 96 点上比 24 点上更清晰；
 - b. 极端值比例：15 分钟粒度下负电价与尖峰的样本占比明显更高。

- c. 业务含义：15 分钟粒度下。“交易时段边界”和“结算时段边界”不再天然对齐(EFM3 用 Hour_business 处理的就是这个问题)

3. 去掉会怎么样：

- 15 分钟数据用 lag24 当“昨天同时刻”->含义变成“6 小时前”，模型学到的是错的而且不会报错
- 24 点数据硬套 lag96：要么列不存在直接失败，要么指到 4 天前，自相关强度断崖下降

3.2.4 真实结果与说明(暂留)

3.3 数据清洗

不做任何清洗，没有插值，没有回填，没有去重，没有异常值处理。只有三种按粒度的分层。详见 3.2.1

带来的后果就是：

- 所有数据的质量问题必须在准备 CSV 文件的时候就解决
- 框架不区分 数据缺失/结构缺失(无成交)/规则缺失(休市)这三类空值

3.4 时间协议

判断任何时间问题前先过三问：

签发时刻是什么？当时真正能拿到那些数据？评价用的是哪段实际值，什么口径？

3.4.1 切分方式

1.解决什么问题：

时间序列里，切分方式直接决定结论可不可靠。随机切分会让模型“见过未来”；错误的边界处理会让测试段开头没有历史可用。

2.怎么解决：

- 目标固定为列配置文件的最后一列(feature_select.py)
- 标准模式下 y 取“未来第 h 步的目标列值”；山西模式下 y 取整日 96 点的目标列
- 三个数据集的目标：天池=price；山西日前=day_ahead_clearing_price；山西实时=realtime_clearing_price

模式	做法	证据
标准模式（天池 / REALE）	按比例连续切分 0.7 / 0.1 / 0.2，不是随机打乱	configs/columns/Tianchi/data_split.json；加载器按比例构造连续边界
山西模式	12 个月度滚动窗，训练窗 1 年，逐月前移	configs/time/data_split_1Y_1M/*.json

切分本身是正确的(时间连续。无随机打乱)，这是时间序列的第一生命线。

3. 去掉会怎么样：

用实时价训练日前模型：指标完全没有业务含义且不会报错

3.4.2 回测形态

- 天池/REALE：单次固定切分，一次训练，一个 test 段
- 山西：月度滚动重训，每月重新训练并预测，更接近生产

3.4.3 与生产的差距

框架没有信息截止点控制——不像 EFM3 那样定义“数据在何时冻结(D14 截止点，dataset_builder.compute_d14_cutoff: 目标日-1 天 14:00)”。在真实生产里，什么时候能用什么数据必须由外部流程保证，框架本身不设防。

3.5 建模与评价

3.5.1 baseline

框架内置的唯一基线是线性回归（Ridge / Lasso / ElasticNet），未定位到 naive 或 seasonal naive 实现。本次实际基线为「Ridge + 前一日同时刻电价 + 同期负荷」。

3.5.2 模型

1.解决什么问题：

电价有两个互相矛盾的特性：强自相关(适合线性/滞后特征)与非线性交互+结构突变(线性模型处理不了，如高负荷+低新能源)。单一模型又很难同时覆盖

2. 怎么解决：

分两个阶段：

- 阶段一（外推）：7 个冻结的时序基础模型——Chronos2、Moirai2、TimerXL、TimesFM2.5、TiRex、Sundial、TabPFN。零样本推理，多采样取均值，输出以 Parquet 缓存并支持增量补算。
- 阶段二（预测）：LightGBM 或线性回归（ridge / lasso / elasticnet），可通过 -- regression_model 三选一。

3.为什么线性模型在这里有竞争力：电价的主结构是强自相关 + 日历周期，滞后项配线性模型就能拿到大部分收益；树模型的价值在于捕捉非线性交互（如“高负荷 + 低新能源”组合），但需要足够的特征与样本，否则过拟合。

4. 去掉会怎么样：

- 去掉阶段一：回到纯滞后回归基线
- 去掉线性选项只留 LightGBM：失去唯一可用的对照基线

3.5.3 训练

每次实验全量重训，无增量或在线学习；随机种子在入口处固定；第二阶段纯 CPU 即可运行。

本次实验结果对比

配置	MAE	MSE	R ²	备注
LightGBM	156.34	44904.6	-0.053	best_iteration = 22169
Ridge (<code>alpha=0.001</code> , 无截距)	128.71	31683.5	0.257	系数: <code>price_lag96</code> = 0.621, <code>demand</code> = 0.0038

三点解读：

1. Ridge 系数 0.621 说明前一日同时刻电价贡献约六成权重——这是电价预测最本质的结构，也是任何新方法必须超越的基线。
2. LightGBM 输给 Ridge 且 R² 为负（不如直接预测均值），是"强自相关 + 特征少 + 跨期分布漂移"下树模型的典型失败模式，恰好印证了论文"回归模型在市场结构突变时样本外失效"的论点。
3. 本次未启用 TSFM，因此这个结果衡量的是**纯滞后回归基线**，不能用来评价混合框架本身。

3.6 工程与生产

- **异常处理**：几乎没有。NaN / Inf 直接抛错，历史窗口含空值则跳过该锚点，，不足 96 点/首点不对/非工作日丢掉整天；没有价格上下限裁剪、没有负电价专门处理、没有模型失败降级——含负价或尖峰的数据上，越界预测会原样输出。
- **模型保存**：模型不落盘，只有 TSFM 输出缓存成 Parquet，回归模型仅保存特征重要性与系数；因此没有模型版本管理，无法追溯"某天的预测出自哪一版模型与数据"。
- **工程流程**：单进程一次性六步串联，带阶段级耗时与内存画像；无调度、无状态、无重试。优点是参数即实验记录，复现成本低；缺点是无法承载每日自动化。
- **部署方式**：仓库内没有任何部署实现（无 daemon、服务、容器编排、CI）。README 声称已在 IoTDB / AINode 上线并给出降本数据，但无法从仓库验证，引用时须标注为作者声明。